

Fine-Tuning & Distillation

An engineering research report on LoRA + knowledge distillation, from
scratch

Sai Veda

Independent Project Research

2025

Contents

- 1. Abstract** 3
- 2. Introduction** 3
 - 2.1 Research Problem 3
 - 2.2 Research Questions and Hypotheses 3
- 3. Technical Background Review** 4
 - 3.1 Design Foundations 4
 - 3.2 Supporting Examples 4
- 4. Research Method** 4
- 5. Data Description** 5
- 6. Analysis** 5
 - 6.1 Benchmark and Validation Results 5
 - 6.2 Test Result Screenshots 5
 - 6.3 Validation Checklist 6
- 7. Discussion** 6
- 8. Conclusion** 7
- 9. Future Work** 7
- 10. References** 7

1. Abstract

This brief presents an adaptation and compression pipeline that implements real fine-tuning, LoRA, and knowledge-distillation mechanics while staying runnable on CPU with a small surrogate model. The final workflow is valuable because it isolates the technique from the hype. It proves what LoRA changes, what distillation recovers, and how a promotion gate should evaluate a challenger before it is accepted. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: LoRA + knowledge distillation, from scratch, implementation evidence, benchmark results, project validation

2. Introduction

The point of the project is not to pretend a giant language model was trained locally. It is to show that the engineering logic of adaptation, parameter efficiency, and promotion gating remains valid when the implementation is simplified honestly. A fully-runnable, GPU-free pipeline that implements the core techniques used to adapt and compress large language models - full fine-tuning, LoRA low-rank adaptation, knowledge distillation, an automated eval/promotion gate, and a synthetic-data scaling study - end to end, deterministically, on a CPU in a few minutes. All numbers below are produced by python scripts/run_pipeline.py and stored in benchmarks/results.json / benchmarks/results.md.

2.1 Research Problem

The point of the project is not to pretend a giant language model was trained locally. It is to show that the engineering logic of adaptation, parameter efficiency, and promotion gating remains valid when the implementation is simplified honestly.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: LoRA 91% params; KD recovers 100% of teacher.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 29 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 The surrogate model keeps the story honest while preserving the real optimization mechanics. This flow mirrors a real adaptation stack: a stronger teacher, a parameter-efficient adapter, and a hard promotion gate.
- 2 LoRA is presented as a parameterization decision with measurable savings, not a branding label.
- 3 Distillation is evaluated on recovered teacher quality rather than anecdotal fluency claims.

3.2 Supporting Examples

- Synthetic instruction dataset and hashed feature pipeline.
- Baseline model, LoRA adapter path, and KD student training.
- Evaluation tables comparing teacher, adapter, and student outcomes.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Chose a small differentiable model so every adaptation technique could run end to end on CPU. Implemented full tuning, LoRA adaptation, and KD with shared evaluation logic. Measured parameter reduction and teacher-student recovery side by side. Used a gate to decide whether a lighter model cleared the required quality bar.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 29 automated tests, 2 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

6.1 Benchmark and Validation Results

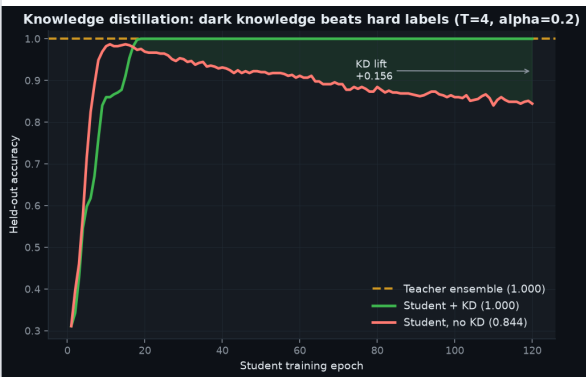
The headline repository result is reported as: LoRA 91% params; KD recovers 100% of teacher. The table below reproduces benchmark evidence included in the repository where available.

Method	Target-domain accuracy	Trainable params
Full fine-tune	0.934	33,417
LoRA (r=4)	0.934	2,889

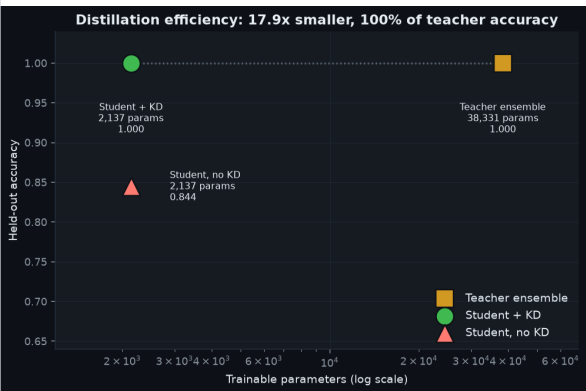
The charts make the parameter-efficiency story visible because savings and quality can be read together. KD recovery matters because compression is only useful if the student preserves enough capability. The evaluation gate turns the project from a training experiment into a release decision workflow.

6.2 Test Result Screenshots

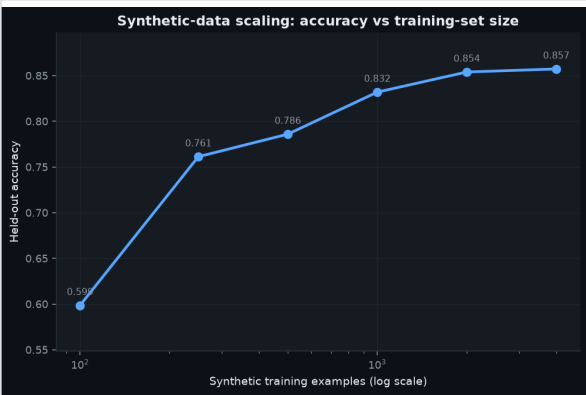
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



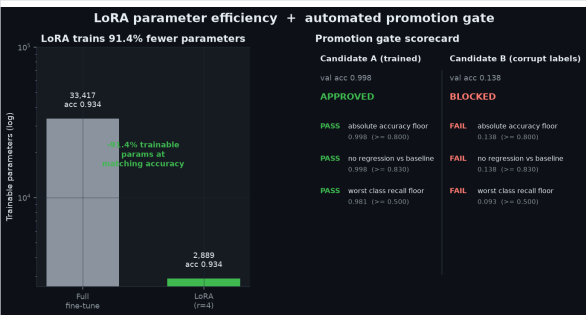
Knowledge distillation: dark knowledge beats hard labels



Distillation efficiency (parameters vs accuracy)



Synthetic-data scaling



LoRA parameter efficiency + promotion-gate scorecard

6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	29 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The final workflow is valuable because it isolates the technique from the hype. It proves what LoRA changes, what distillation recovers, and how a promotion gate should evaluate a challenger before it is accepted. The charts make the parameter-efficiency story visible because savings and quality can be read together. KD recovery matters because

compression is only useful if the student preserves enough capability. The evaluation gate turns the project from a training experiment into a release decision workflow. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Fine-Tuning & Distillation is best understood as a complete technical implementation supported by repository-native evidence. This brief presents an adaptation and compression pipeline that implements real fine-tuning, LoRA, and knowledge-distillation mechanics while staying runnable on CPU with a small surrogate model.

9. Future Work

- Swap the surrogate model for a transformer implementation when GPU resources are available.
- Add richer task slices so promotion decisions reflect multiple instruction categories.
- Track adapter merge behavior and inference cost under a serving-oriented benchmark.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/generator_throughput.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/lora.csv - benchmark or result artefact used in the analysis section.
5. benchmarks/results.json - benchmark or result artefact used in the analysis section.
6. benchmarks/results.md - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/01_kd_curves.png - generated screenshot evidence included in the report.